

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TRUNG ƯƠNG**



KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

Học phần: Kiểm Thử Phần Mềm

Đề tài: Hệ thống Web quản lý công việc

Giảng viên: Th.S Vi Mạnh Hùng

Sinh viên thực hiện:

Phạm Đức Cường - CT230035

Nguyễn Xuân Đông - CT230030

Nguyễn Tiến Dũng - CT230037

Lớp: CT12302

Năm 2026

Mục Lục

DANH MỤC TỪ VIẾT TẮT	3
BANH MỤC BẢNG BIỂU	5
DANH MỤC HÌNH VẼ	6
 CHƯƠNG 1: TỔNG QUAN VỀ DỰ ÁN VÀ YÊU CẦU CHẤT LƯỢNG.....	 7
1.1. Giới thiệu đề tài.....	7
1.2. Mô tả bài toán / chức năng	7
Bảng 1.1 – Ma trận Actor và Use case	8
1.3. Mục tiêu & Phạm vi kiểm thử	9
1.3.1. Mục tiêu kiểm thử	9
1.3.2. Phạm vi kiểm thử (In-scope)	9
1.3.3. Ngoài phạm vi kiểm thử (Out-of-scope).....	10
 CHƯƠNG 2: KẾ HOẠCH ĐẢM BẢO CHẤT LƯỢNG (SQA PLAN) & KIỂM THỬ (TEST PLAN).....	 11
2.1. Kế hoạch và Quy trình SQA (Hoạt động Review).....	11
2.1.1. Quy trình review hợp đồng và yêu cầu.....	11
2.1.2. Quy trình review tài liệu đặc tả chức năng (SRS).....	11
Bảng 2.1 – Checklist Review tài liệu đặc tả (SRS).....	11
2.1.3. Quy trình review mã nguồn (Code Review)	12
Bảng 2.2 – Checklist Code Review	12
2.2. Chiến lược kiểm thử (Test Strategy).....	12
2.3. Môi trường và Công cụ kiểm thử.....	13
Bảng 2.3 – Môi trường và công cụ sử dụng.....	13
2.4. Tiêu chí dừng kiểm thử (Exit Criteria).....	13
2.5. Chuẩn bị dữ liệu kiểm thử (Test Data Preparation).....	14
 CHƯƠNG 3: THIẾT KẾ KIỂM THỬ & KẾT QUẢ HOẠT ĐỘNG REVIEW (STATIC TESTING).....	 15
3.1. Kết quả thực hiện Hoạt động Review (SQA).....	15
Bảng 3.1 – Biên bản kết quả Review (Static QA Findings)	15
3.2. Thiết kế Ca kiểm thử (Test Case Design).....	16
Bảng 3.2 – Danh sách Test Case tiêu biểu (Module Đăng nhập).....	17
Bảng 3.3 – Danh sách Test Case tiêu biểu (Module Quản lý công việc). 17	
Bảng 3.4 – Danh sách Test Case tiêu biểu (Module Quản lý người dùng & Phân quyền).....	18
Bảng 3.5 – Danh sách Test Case tiêu biểu (Dashboard & Giao diện).....	19
Bảng 3.6 – Danh sách Test Case bổ sung (Kiểm thử hộp trắng & Dữ liệu biên).....	20
3.3. Ưu tiên hoá kiểm thử theo rủi ro (Risk-based Test Prioritization).....	21
Bảng 3.7 – Ma trận ưu tiên kiểm thử theo rủi ro	21
 CHƯƠNG 4: THỰC THI KIỂM THỬ & QUẢN LÝ LỖI (DYNAMIC TESTING).....	 22
4.1. Kết quả thực thi Test Case.....	22
Bảng 4.1 – Tổng hợp kết quả thực thi Test Case theo module	22
4.2. Nhật ký lỗi (Bug Log) và Quản lý lỗi	23
Bảng 4.2 – Nhật ký lỗi (Bug Log)	24

4.2.1. Minh chứng chi tiết các lỗi mức độ Critical/Major.....	25
BUG-01 — Sai số liệu phân trang khi kết hợp bộ lọc trạng thái	25
BUG-02 — Trạng thái công việc bị ghi đè âm thầm theo tiến độ	26
BUG-03 — USER có thể can thiệp tham số userId khi lưu task	26
BUG-04 — Không có bảo vệ CSRF cho các form POST	27
BUG-05- Server không xác thực nội dung thực tế của file avatar tải lên	28
4.3. So sánh kết quả kiểm thử giữa các vòng (Round 1 và Round 2).....	28
Bảng 4.3 – So sánh kết quả kiểm thử giữa Round 1 và Round 2.....	29
CHƯƠNG 5: ĐÁNH GIÁ & KẾT LUẬN	30
5.1. Đánh giá chất lượng phần mềm.....	30
Bảng 5.1 – Đối chiếu kết quả với Exit Criteria	30
5.2. Kết quả nhóm đạt được	30
5.2.1. Về mặt sản phẩm.....	31
5.2.2. Về mặt kỹ năng	31
5.3. Hạn chế & Hướng phát triển	31
5.3.1. Hạn chế.....	31
5.3.2. Hướng phát triển	32
Phụ lục	32

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
SQA	Software Quality Assurance – Đảm bảo chất lượng phần mềm
SRS	Software Requirement Specification – Tài liệu đặc tả yêu cầu phần mềm
TC	Test Case – Ca kiểm thử

UI/UX	User Interface / User Experience – Giao diện / Trải nghiệm người dùng
CRUD	Create – Read – Update – Delete (Thêm – Xem – Sửa – Xóa)
MVC	Model – View – Controller (mô hình kiến trúc phần mềm)
JPA	Java Persistence API
ORM	Object-Relational Mapping
CSRF	Cross-Site Request Forgery – Giả mạo yêu cầu liên trang
BVA	Boundary Value Analysis – Phân tích giá trị biên
EP	Equivalence Partitioning – Phân vùng tương đương

DANH MỤC BẢNG BIỂU

STT	Giải thích
1.1	Ma trận phân quyền chức năng theo vai trò
2.1	Checklist Review tài liệu đặc tả (SRS)
2.2	Checklist Code Review
2.3	Môi trường và công cụ sử dụng
3.1	Biên bản kết quả Review (Static QA Findings)
3.2	Danh sách Test Case tiêu biểu (Module Đăng nhập)
3.3	Danh sách Test Case tiêu biểu (Module Quản lý công việc)
3.4	Danh sách Test Case tiêu biểu (Module Quản lý người dùng & Phân quyền)
3.5	Danh sách Test Case tiêu biểu (Dashboard & Giao diện)
3.6	Danh sách Test Case bổ sung (Kiểm thử hộp trắng & Dữ liệu biên)
3.7	Ma trận ưu tiên kiểm thử theo rủi ro
4.1	Tổng hợp kết quả thực thi Test Case theo module
4.2	Nhật ký lỗi (Bug Log)
4.3	So sánh kết quả kiểm thử giữa Round 1 và Round 2
5.1	Đối chiếu kết quả với Exit Criteria

DANH MỤC HÌNH VẼ

Hình	Giải thích
1.1	Sơ đồ Use Case tổng quát của hệ thống Task Manager
4.1	Biểu đồ tỷ lệ kết quả thực thi Test Case (Pass/Fail/Blocked)
4.2	So sánh số lượng Test Case và số lượng Test Case Pass theo module
4.3	Phân bố lỗi được ghi nhận theo mức độ nghiêm trọng (Severity)

CHƯƠNG 1: TỔNG QUAN VỀ DỰ ÁN VÀ YÊU CẦU CHẤT LƯỢNG

1.1. Giới thiệu đề tài

Task Manager là hệ thống quản lý công việc dạng web, được xây dựng nhằm hỗ trợ cá nhân và nhóm làm việc theo dõi, phân công và cập nhật tiến độ các đầu việc hằng ngày trong một tổ chức có quy mô nhỏ. Hệ thống được phát triển theo kiến trúc MVC (Model – View – Controller) trên nền tảng Java Spring Boot, sử dụng Thymeleaf để render giao diện phía server và MySQL làm hệ quản trị cơ sở dữ liệu quan hệ.

Về mặt nghiệp vụ, hệ thống phục vụ hai nhóm người dùng chính: quản trị viên (ADMIN) và nhân viên (USER). Quản trị viên có toàn quyền quản lý tài khoản người dùng, xem và chỉnh sửa toàn bộ công việc trong hệ thống; trong khi người dùng thông thường chỉ được thao tác trên các công việc được giao cho chính mình. Ngoài chức năng nghiệp vụ cốt lõi là quản lý công việc (Task) và quản lý người dùng (User), hệ thống còn cung cấp một trang Dashboard tổng quan, hiển thị số liệu thống kê trực quan bằng biểu đồ (sử dụng thư viện Chart.js) về số lượng công việc theo từng trạng thái.

Báo cáo này được thực hiện trong khuôn khổ môn học Đảm bảo chất lượng và Kiểm thử phần mềm (Software Quality Assurance – SQA), với mục tiêu áp dụng các kiến thức về quy trình đảm bảo chất lượng tĩnh (Static QA – review tài liệu, review mã nguồn) và kiểm thử động (Dynamic Testing – thiết kế và thực thi ca kiểm thử) lên một sản phẩm phần mềm thực tế do chính nhóm phát triển, từ đó đưa ra đánh giá khách quan về chất lượng sản phẩm trước khi bàn giao.

1.2. Mô tả bài toán / chức năng

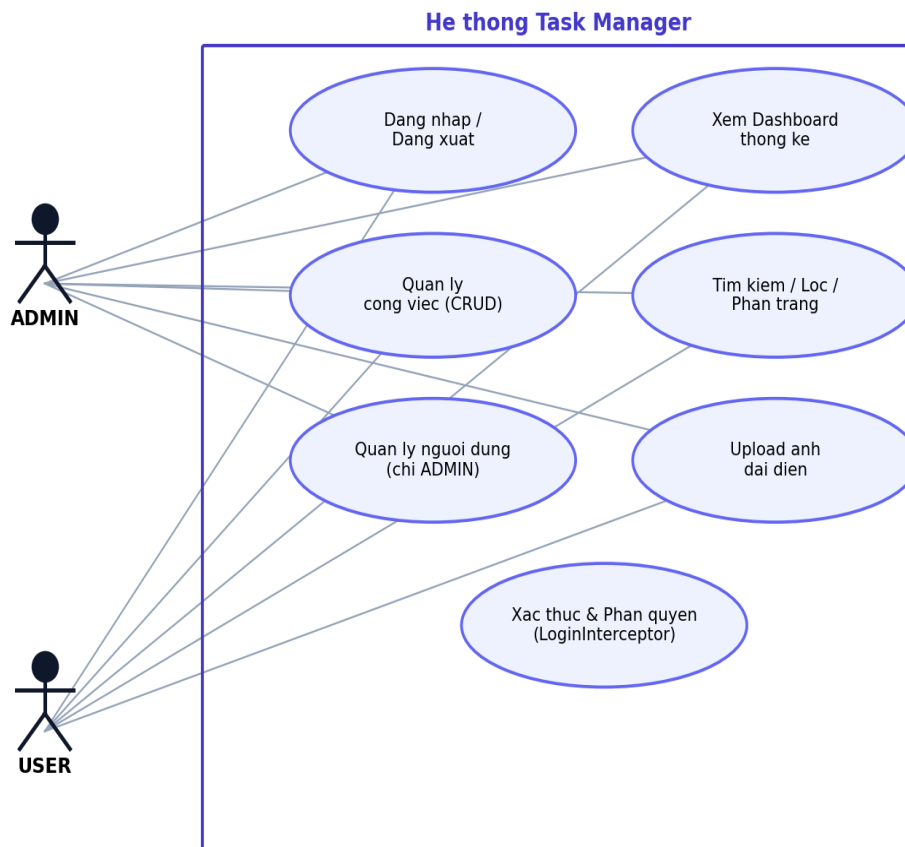
Hệ thống Task Manager gồm hai tác nhân (actor) tương tác trực tiếp: Quản trị viên (ADMIN) và Người dùng (USER). Sơ đồ ca sử dụng (Use Case) tổng quát của hệ thống được mô tả bằng bảng ảnh xạ Actor – Use case dưới đây, do công cụ soạn thảo văn bản không hỗ trợ nhúng sơ đồ UML dạng vector; nhóm đề nghị xem sơ đồ Use Case dạng hình ảnh chi tiết tại Chương 3 – tài liệu đặc tả (SRS) đính kèm phụ lục.

Bảng 1.1 – Ma trận Actor và Use case

STT	Use case	ADMIN	USER
1	Đăng nhập / Đăng xuất (UC-01)	X	X
2	Xem Dashboard thống kê (UC-02)	X	X
3	Xem danh sách công việc (UC-03)	X (tất cả)	X (của bản thân)
4	Thêm mới công việc (UC-04)	X	X
5	Sửa công việc (UC-05)	X (tất cả)	X (của bản thân)
6	Xoá công việc (UC-06)	X (tất cả)	X (của bản thân)
7	Tìm kiếm / Lọc / Phân trang công việc (UC-07)	X	X
8	Quản lý danh sách người dùng (UC-08)	X	—
9	Thêm / Sửa / Xoá người dùng, upload avatar (UC-09)	X	—
10	Tìm kiếm, phân trang người dùng (UC-10)	X	—

Bảng 1.1: Ma trận phân quyền chức năng theo vai trò, trích xuất từ LoginInterceptor.java và các Controller

Để trực quan hoá mối quan hệ giữa hai tác nhân và các nhóm chức năng chính, nhóm xây dựng sơ đồ Use Case tổng quát như Hình 1.1. Sơ đồ được vẽ lại dựa trên việc phân tích trực tiếp các endpoint và điều kiện phân quyền trong mã nguồn (không có sẵn tài liệu UML gốc từ giai đoạn phân tích thiết kế ban đầu của dự án), do đó cũng được xem là một phát hiện review được ghi nhận tại Chương 3 (thiếu tài liệu thiết kế UML).



Hình 1.1: Sơ đồ Use Case tổng quát của hệ thống Task Manager

Các luồng nghiệp vụ chính cần kiểm thử được xác định dựa trên việc phân tích mã nguồn các lớp Controller của hệ thống (LoginController, TaskController, UserController, HomeController), cụ thể như sau:

Luồng đăng nhập: người dùng nhập username/password → hệ thống xác thực bằng BCryptPasswordEncoder, so khớp với bản ghi trong bảng users → nếu hợp lệ, lưu đối tượng User vào HttpSession và chuyển hướng tới trang chủ; ngược lại hiển thị thông báo lỗi "Sai tên đăng nhập hoặc mật khẩu!".

Luồng kiểm soát truy cập: lớp LoginInterceptor chặn mọi request (trừ /login, /logout, tài nguyên tĩnh) khi session chưa có thuộc tính user; đồng thời chặn riêng nhóm đường dẫn /users nếu vai trò người dùng khác ADMIN.

Luồng quản lý công việc: tạo/sửa task với các trường title, description, status, deadline, priority, progress, người thực hiện (userId); trạng thái công việc được đồng bộ tự động theo phần trăm tiến độ (progress = 0 → "Chưa

làm", progress = 100 → "Hoàn thành", còn lại → "Đang làm") ngay trong TaskController.saveTask().

Luồng quản lý người dùng: tạo/sửa user kèm upload ảnh đại diện (MultipartFile), mã hoá mật khẩu bằng BCrypt, giới hạn dung lượng file tải lên tối đa 10MB theo cấu hình spring.servlet.multipart trong application.properties.

Luồng thông kê Dashboard: HomeController truy vấn số lượng task theo từng trạng thái (countByStatus) và tổng số user, hiển thị dưới dạng thẻ số liệu (stat-card) và biểu đồ tròn (doughnut chart) bằng Chart.js.

1.3. Mục tiêu & Phạm vi kiểm thử

1.3.1. Mục tiêu kiểm thử

Mục tiêu của hoạt động kiểm thử là đánh giá mức độ đáp ứng yêu cầu chức năng và một phần yêu cầu phi chức năng (giao diện, khả năng sử dụng) của hệ thống Task Manager trước khi xem xét bàn giao; phát hiện sớm các sai lệch giữa hành vi thực tế của hệ thống so với đặc tả nghiệp vụ; đồng thời cung cấp số liệu định lượng (tỷ lệ pass/fail, số lượng và mức độ nghiêm trọng của lỗi) làm cơ sở cho quyết định go/no-go khi kết thúc dự án.

1.3.2. Phạm vi kiểm thử (In-scope)

Kiểm thử chức năng (Functional Testing) ở cấp độ System Testing và Integration Testing thông qua giao diện web (kiểm thử hộp đen) cho toàn bộ các module: Đăng nhập/Đăng xuất, Quản lý công việc (CRUD), Quản lý người dùng (CRUD, upload avatar), Tìm kiếm – Lọc – Phân trang, Dashboard thống kê.

Kiểm thử phân quyền truy cập (Authorization Testing) giữa hai vai trò ADMIN và USER dựa trên LoginInterceptor và logic kiểm tra vai trò trong từng Controller.

Kiểm thử hộp trắng (White-box Testing) ở mức đơn vị cho một số hàm nghiệp vụ quan trọng có nhiều nhánh điều kiện, cụ thể là phương thức TaskController.saveTask() (logic đồng bộ progress → status) và các callback @PrePersist/@PreUpdate trong entity Task.

Kiểm thử giao diện (UI Testing) ở mức cơ bản: kiểm tra hiển thị responsive, chế độ Dark Mode (theme.js, dark.css), tính đúng đắn của các thông báo lỗi trên form.

Review tĩnh (Static QA): review tài liệu đặc tả yêu cầu (SRS) và review mã nguồn (Code Review) theo checklist được xây dựng ở Chương 2.

1.3.3. Ngoài phạm vi kiểm thử (Out-of-scope)

Kiểm thử bảo mật chuyên sâu (Penetration Testing) như SQL Injection nâng cao, tấn công CSRF/XSS có chủ đích, rà quét lỗ hổng tự động bằng công cụ chuyên dụng (OWASP ZAP, Burp Suite) — nhóm chỉ ghi nhận các quan sát bảo mật phát hiện được trong quá trình review và kiểm thử chức năng thông thường.

Kiểm thử hiệu năng và khả năng chịu tải (Load/Stress/Performance Testing) với số lượng người dùng đồng thời lớn, do giới hạn về thời gian, hạ tầng máy chủ thử nghiệm và không nằm trong mục tiêu môn học.

Kiểm thử tương thích đa trình duyệt/đa thiết bị toàn diện (Cross-browser/Cross-device Compatibility Testing) — chỉ kiểm thử trên trình duyệt Google Chrome ở độ phân giải desktop phổ biến.

Kiểm thử tự động hoá end-to-end (Selenium, Cypress...) và kiểm thử API độc lập bằng Postman/JMeter, vì hệ thống được xây dựng theo mô hình server-side rendering (Thymeleaf) không tách API riêng biệt.

Kiểm thử bản địa hoá đa ngôn ngữ (i18n) — hệ thống chỉ hỗ trợ tiếng Việt.

CHƯƠNG 2: KẾ HOẠCH ĐẢM BẢO CHẤT LƯỢNG (SQA PLAN) & KIỂM THỬ (TEST PLAN)

2.1. Kế hoạch và Quy trình SQA (Hoạt động Review)

Song song với hoạt động kiểm thử động, nhóm triển khai các hoạt động đảm bảo chất lượng tĩnh (Static QA) nhằm phát hiện lỗi ngay từ giai đoạn đặc tả và lập trình, trước khi hệ thống được đưa vào chạy thử. Ba hoạt động review chính được thực hiện gồm: review hợp đồng/yêu cầu ban đầu, review tài liệu đặc tả chức năng (SRS) và review mã nguồn (Code Review).

2.1.1. Quy trình review hợp đồng và yêu cầu

Trước khi bắt đầu triển khai, nhóm đối chiếu README.md và các trao đổi yêu cầu ban đầu với danh sách chức năng thực tế đã liệt kê (đăng ký/đăng nhập, quản lý công việc, quản lý người dùng). Kết quả review cho thấy tài liệu README có đề cập chức năng "Đăng ký tài khoản" nhưng mã nguồn không có Controller hay endpoint nào xử lý việc tự đăng ký (chỉ có LoginController xử lý đăng nhập/đăng xuất, tài khoản chỉ được ADMIN tạo qua /users/new) — đây là một sai lệch giữa tài liệu và sản phẩm được ghi nhận trong biên bản review.

2.1.2. Quy trình review tài liệu đặc tả chức năng (SRS)

Tài liệu đặc tả được review theo hình thức Walkthrough giữa các thành viên trong nhóm, tập trung vào tính đầy đủ, nhất quán và khả năng kiểm thử được (testability) của từng yêu cầu. Checklist review SRS được xây dựng dựa trên tiêu chuẩn IEEE 830 rút gọn, áp dụng cho từng chức năng của hệ thống.

Bảng 2.1 – Checklist Review tài liệu đặc tả (SRS)

STT	Tiêu chí đánh giá	Đạt/ Không đạt	Ghi chú
1	Yêu cầu có được mô tả rõ ràng, không mập mờ (unambiguous)?	Không đạt	Chưa quy định rõ giá trị status hợp lệ ngoài 3 giá trị mặc định
2	Yêu cầu có nhất quán giữa các phần đặc tả?	Không đạt	README ghi "Đăng ký tài khoản" nhưng không có trong đặc tả chi tiết luồng
3	Yêu cầu có thể kiểm thử được (testable)?	Đạt	Đa số yêu cầu có tiêu chí

STT	Tiêu chí đánh giá	Đạt/ Không đạt	Ghi chú
			input/output rõ ràng
4	Yêu cầu phi chức năng (hiệu năng, bảo mật) có được nêu?	Không đạt	Không có yêu cầu cụ thể về thời gian phản hồi, chính sách mật khẩu
5	Ràng buộc dữ liệu (data constraint) có đầy đủ?	Một phần	Có ràng buộc NOT NULL ở entity nhưng thiếu ràng buộc độ dài mật khẩu tối thiểu
6	Có ma trận truy vết yêu cầu – chức năng (traceability)?	Không đạt	Chưa xây dựng, nhóm bổ sung ở Bảng 1.1

Bảng 2.1: Kết quả checklist review SRS – 1/6 tiêu chí đạt hoàn toàn

2.1.3. Quy trình review mã nguồn (Code Review)

Hoạt động Code Review được thực hiện thủ công (manual review) theo hình thức Peer Review, mỗi thành viên đọc và đối chiếu mã nguồn module không do mình viết, sử dụng checklist tổng quát bao gồm các nhóm tiêu chí: tính đúng đắn logic nghiệp vụ, xử lý ngoại lệ, bảo mật, chuẩn coding convention và khả năng bảo trì. Kết quả chi tiết của hoạt động review mã nguồn (bao gồm danh sách các điểm bất hợp lý phát hiện được) được trình bày đầy đủ tại mục 3.1 của báo cáo này.

Bảng 2.2 – Checklist Code Review

Nhóm tiêu chí	Nội dung kiểm tra
Logic nghiệp vụ	Điều kiện rẽ nhánh có bao phủ đầy đủ các trường hợp? Có tồn tại logic mâu thuẫn giữa các lớp (VD: giá trị nhập từ UI có bị ghi đè bởi Controller không)?
Xử lý ngoại lệ	Có try-catch hợp lý? Lỗi có được log/ thông báo cho người dùng hay bị nuốt (swallow exception) bằng e.printStackTrace()?
Bảo mật	Mật khẩu có được mã hoá? Thông tin nhạy cảm (DB password) có bị hard-code? Có kiểm soát quyền truy cập ở tầng Controller (không chỉ dựa vào ẩn menu trên giao diện)?
Coding convention	Đặt tên biến/hàm có nhất quán? Có đoạn code trùng lặp (duplicate) giữa các Controller có thể refactor?
CSS/Front-end	Biến CSS (custom property) có được định nghĩa và tham chiếu đúng? Style có xung đột giữa các file dùng chung class name?

2.2. Chiến lược kiểm thử (Test Strategy)

Nhóm áp dụng kết hợp ba phương pháp kiểm thử nhằm tối ưu độ bao phủ trong điều kiện thời gian và nguồn lực hạn chế của một đồ án môn học:

Kiểm thử hộp đen (Black-box Testing): áp dụng cho phần lớn các ca kiểm thử chức năng và giao diện, dựa trên đặc tả yêu cầu mà không quan tâm đến cấu trúc mã nguồn bên trong. Các kỹ thuật thiết kế ca kiểm thử cụ thể được trình bày tại mục 3.2.

Kiểm thử hộp trắng (White-box Testing): áp dụng cho các hàm xử lý nghiệp vụ có nhiều nhánh điều kiện quan trọng, tiêu biểu là logic đồng bộ progress → status trong TaskController.saveTask() và các callback vòng đời entity (@PrePersist, @PreUpdate) của lớp Task, nhằm đảm bảo bao phủ tất cả các nhánh (branch coverage) của điều kiện if/else if/else.

Kiểm thử hộp xám (Gray-box Testing): áp dụng cho nhóm kiểm thử phân quyền — tester vừa dựa vào giao diện (menu bị ẩn với USER) vừa dựa vào hiểu biết về mã nguồn (LoginInterceptor, kiểm tra role trong Controller) để chủ động thử truy cập trực tiếp bằng URL vào các chức năng lẽ ra không được hiển thị, phát hiện các lỗi hỏng kiểm soát truy cập tại tầng Controller mà giao diện không thể hiện.

2.3. Môi trường và Công cụ kiểm thử

Bảng 2.3 – Môi trường và công cụ sử dụng

Hạng mục	Chi tiết
Hệ điều hành máy chủ / máy test	Windows 11 (môi trường phát triển và kiểm thử cục bộ)
Ngôn ngữ / Nền tảng	Java 17, Spring Boot 3.5.0, Spring MVC, Spring Data JPA, Thymeleaf
Cơ sở dữ liệu	MySQL 8.x, kết nối qua mysql-connector-j, schema tự sinh bằng Hibernate (ddl-auto=update)
Công cụ build	Apache Maven (thông qua Maven Wrapper – mvnw/mvnw.cmd)
IDE phát triển	IntelliJ IDEA
Trình duyệt kiểm thử	Google Chrome (phiên bản mới nhất tại thời điểm kiểm thử)
Công cụ quản lý test case & lỗi (Bug Tracking)	Bảng tính Excel/Google Sheets dùng chung cho nhóm (do quy mô dự án nhỏ, không triển khai Jira/Redmine)
Công cụ hỗ trợ kiểm thử	DevTools của Chrome (kiểm tra request/response, responsive), MySQL Workbench (đối chiếu dữ liệu sau thao tác CRUD)
Công cụ quản lý mã nguồn	Git & GitHub (repository DevCuong205/baiTapNho)

2.4. Tiêu chí dừng kiểm thử (Exit Criteria)

Việc kiểm thử được xem là hoàn tất và có thể tiến hành đánh giá bàn giao khi thoả mãn đồng thời các điều kiện sau:

100% số ca kiểm thử (test case) đã được thiết kế trong phạm vi (in-scope) đã được thực thi ít nhất một lần.

Tỷ lệ ca kiểm thử Pass đạt tối thiểu 90% trên tổng số ca kiểm thử đã thực thi.

Không còn lỗi ở mức độ nghiêm trọng Critical đang ở trạng thái mở (Open) chưa được xử lý hoặc có phương án xử lý tạm thời (workaround) được các bên thống nhất.

Toàn bộ lỗi mức độ Major được ghi nhận đầy đủ trong Bug Log kèm mức độ ưu tiên xử lý, không nhất thiết phải fix 100% trước khi bàn giao nhưng phải được liệt kê minh bạch trong Chương 5 (Hạn chế).

Các phát hiện từ hoạt động review tĩnh (Chương 3, mục 3.1) đã được lập thành biên bản và thông báo tới nhóm phát triển.

Trong trường hợp không thể đạt đủ các tiêu chí trên do giới hạn thời gian của đề án, nhóm sẽ đưa ra khuyến nghị bàn giao có điều kiện (conditional release) kèm danh sách lỗi tồn đọng, được trình bày chi tiết tại Chương 5.

2.5. Chuẩn bị dữ liệu kiểm thử (Test Data Preparation)

Trước khi thực thi các ca kiểm thử, nhóm chuẩn bị một tập dữ liệu mẫu (seed data) được nạp trực tiếp vào MySQL thông qua Hibernate (ddl-auto=update tự sinh schema, dữ liệu mẫu được insert thủ công qua MySQL Workbench) nhằm đảm bảo môi trường kiểm thử có đủ các trường hợp biên cần thiết mà không phụ thuộc hoàn toàn vào dữ liệu phát sinh ngẫu nhiên trong lúc thao tác thủ công. Bộ dữ liệu mẫu bao gồm:

2 tài khoản với vai trò ADMIN và 4 tài khoản vai trò USER, trong đó có 1 tài khoản USER chưa được gán bất kỳ công việc nào (phục vụ kiểm thử trường hợp danh sách rỗng — empty-cell trong tasks.html/users.html).

12 công việc (task) được rải đều ở cả ba trạng thái Chưa làm / Đang làm / Hoàn thành, trong đó có ít nhất 2 task với deadline trong quá khứ và status khác "Hoàn thành" để kiểm thử cảnh báo quá hạn (overdue-badge trong tasks.html).

Ít nhất 1 task không gán người thực hiện (user = null) để kiểm thử hiển thị nhãn "Chưa gán" trên giao diện danh sách công việc.

Dữ liệu username trùng lặp (thử tạo user mới với username đã tồn tại) nhằm kiểm thử ràng buộc @Column(unique = true) ở tầng CSDL có được xử lý và phản hồi lỗi hợp lý cho người dùng hay không, thay vì để lộ lỗi hệ thống (stack trace) trực tiếp ra giao diện.

Việc chuẩn bị dữ liệu mẫu có kiểm soát giúp nhóm tái lập (reproduce) các lỗi một cách nhất quán giữa các thành viên, đặc biệt hữu ích khi xác minh lại lỗi BUG-01 (liên quan phân trang kết hợp bộ lọc) — lỗi này chỉ bộc lộ rõ khi số lượng bản ghi trong CSDL đủ lớn để tạo ra nhiều hơn một trang dữ liệu.

CHƯƠNG 3: THIẾT KẾ KIỂM THỬ & KẾT QUẢ HOẠT ĐỘNG REVIEW (STATIC TESTING)

3.1. Kết quả thực hiện Hoạt động Review (SQA)

Thông qua hoạt động review tài liệu và review mã nguồn được mô tả tại Chương 2, nhóm đã phát hiện tổng cộng 12 điểm bất hợp lý/nhập nhằng trong đặc tả và cài đặt trước khi tiến hành kiểm thử động. Việc phát hiện sớm các vấn đề này giúp tiết kiệm chi phí sửa lỗi đáng kể so với việc phát hiện ở giai đoạn kiểm thử hệ thống hoặc sau khi bàn giao. Bảng 3.1 tổng hợp toàn bộ các phát hiện, được phân loại theo nhóm: Đặc tả (SRS), Logic nghiệp vụ, Bảo mật, và Front-end/CSS.

Bảng 3.1 – Biên bản kết quả Review (Static QA Findings)

#	Hạng mục review	Vị trí trong mã nguồn / tài liệu	Mô tả phát hiện	Mức độ
1	Đặc tả (SRS)	README.md – mục Chức năng	README liệt kê chức năng "Đăng ký tài khoản" nhưng không tồn tại Controller/endpoint xử lý tự đăng ký; tài khoản chỉ được ADMIN khởi tạo qua /users/new.	Trung bình
2	Đặc tả (SRS)	README.md – mục Công nghệ sử dụng	README ghi công nghệ "Bootstrap 5" nhưng pom.xml và các file HTML không tham chiếu Bootstrap CSS/JS, giao diện dùng CSS thuần tự viết.	Thấp
3	Logic nghiệp vụ	TaskController.saveTask()	Trường "Trạng thái" do người dùng chọn trên form task-form.html luôn bị ghi đè bởi logic tự động suy ra từ progress; UI cho phép chọn Trạng thái nhưng giá trị thực tế không phụ thuộc lựa chọn đó khi progress khác 0 và 100 theo một cách không tường minh với người dùng.	Cao
4	Logic nghiệp vụ	TaskController.listTasks()	Bộ lọc theo status/priority được áp dụng bằng Stream sau khi đã phân trang (Pageable) ở tầng Repository, dẫn đến số liệu totalTasks/totalPages hiển thị không khớp với danh sách thực tế sau khi lọc.	Cao
5	Bảo mật	application.properties	Mật khẩu kết nối cơ sở dữ liệu (spring.datasource.password) được lưu dạng plaintext trực tiếp trong file	Cao

#	Hạng mục review	Vị trí trong mã nguồn / tài liệu	Mô tả phát hiện	Mức độ
			cấu hình, không sử dụng biến môi trường hay secret store.	
6	Bảo mật	application.properties	Chuỗi kết nối JDBC sử dụng useSSL=false, dữ liệu truyền giữa ứng dụng và MySQL không được mã hoá trên đường truyền.	Trung bình
7	Bảo mật	pom.xml / SecurityConfig.java	Dự án chỉ khai báo spring-security-crypto (chỉ dùng BCryptPasswordEncoder) mà không sử dụng spring-boot-starter-security đầy đủ, nên không có sẵn cơ chế bảo vệ CSRF cho các form POST (xóa, sửa, lưu task/user).	Cao
8	Bảo mật	UserController.saveUser()	Không kiểm tra loại nội dung (content-type) thực tế của file avatar tải lên ở phía server, chỉ giới hạn thông qua thuộc tính accept="image/*" ở phía client (có thể bị bỏ qua).	Trung bình
9	Bảo mật	TaskController.saveTask()	Không giới hạn giá trị tham số userId theo vai trò người gửi request: về mặt lý thuyết một USER có thể chỉnh sửa request để gán task cho user khác nếu can thiệp vào form.	Trung bình
10	Front-end/CSS	style.css, tasks.css, dashboard.css, users.css, user-form.css, task-form.css, login.css	Biến CSS --text-secondary được định nghĩa tự tham chiếu chính nó (--text-secondary: var(--text-secondary);) tại 6/7 file CSS, khiến giá trị biến không hợp lệ (invalid at computed value time), trình duyệt áp dụng giá trị kế thừa mặc định thay vì màu xám thiết kế.	Trung bình
11	Front-end/CSS	tasks.css – khối .task-table thead th (dark mode)	Thừa một dấu đóng ngoặc nhọn "}" sau khối CSS body.dark .task-table thead th, gây lỗi cú pháp cục bộ trong stylesheet.	Thấp
12	Kiểm thử tự động	TaskmanagerApplicationTests.java	Dự án chỉ có duy nhất 1 unit test mặc định (contextLoads – kiểm tra Spring Context khởi động thành công), chưa có bất kỳ unit test nào cho tầng Service/Repository/logic nghiệp vụ.	Cao

Bảng 3.1: 12 phát hiện từ hoạt động review tĩnh, trong đó 5 phát hiện ở mức Cao cần được nhóm phát triển xử lý trước khi phát hành chính thức

Các phát hiện số 3, 4 và 12 được nhóm ưu tiên xác nhận lại bằng kiểm thử động (Dynamic Testing) ở Chương 4 để đánh giá mức độ ảnh hưởng thực tế đến người dùng cuối, do đây là các lỗi liên quan trực tiếp đến tính đúng đắn của số liệu và dữ liệu hiển thị.

3.2. Thiết kế Ca kiểm thử (Test Case Design)

Đối với các trường dữ liệu đầu vào dạng văn bản/số (title, progress, deadline, username, password...), nhóm áp dụng kỹ thuật Phân vùng tương đương (Equivalence Partitioning) kết hợp Phân tích giá trị biên (Boundary Value Analysis), ví dụ với trường progress (thanh trượt 0–100, bước nhảy 10): các giá trị biên 0, 10, 90, 100 và giá trị ngoài khoảng được kiểm thử riêng biệt để xác nhận đúng ánh xạ sang trạng thái tương ứng. Đối với các nghiệp vụ có nhiều điều kiện kết hợp (ví dụ: xác định trạng thái hiển thị của task dựa trên deadline và status để cảnh báo quá hạn trong tasks.html), nhóm sử dụng Bảng quyết định (Decision Table) để đảm bảo bao phủ đầy đủ các tổ hợp điều kiện. Với các hàm nghiệp vụ hộp trắng (saveTask, prePersist/preUpdate), kỹ thuật kiểm thử luồng điều khiển (Control Flow Testing) được dùng để thiết kế ca kiểm thử bao phủ mọi nhánh rẽ if/else if/else.

Do số lượng ca kiểm thử chi tiết cho toàn hệ thống khá lớn (53 ca kiểm thử được thiết kế đầy đủ trong file Excel quản lý test case của nhóm), báo cáo chỉ trình bày các ca kiểm thử tiêu biểu/đại diện cho từng module chức năng. Bộ test case đầy đủ được đính kèm tại Phụ lục A (file Excel "TaskManager_TestCases.xlsx").

Bảng 3.2 – Danh sách Test Case tiêu biểu (Module Đăng nhập)

Mã TC	Mô tả	Dữ liệu vào	Kết quả mong đợi
TC_LG_01	Đăng nhập với tài khoản và mật khẩu hợp lệ (role ADMIN)	username=admin, password đúng	Đăng nhập thành công, chuyển hướng về Dashboard ("/"), session lưu user
TC_LG_02	Đăng nhập với mật khẩu sai	username=admin, password sai	Hiển thị thông báo "Sai tên đăng nhập hoặc mật khẩu!", ở lại trang login
TC_LG_03	Đăng nhập với username không tồn tại	username=abcxyz	Hiển thị thông báo lỗi tương tự TC_LG_02 (không tiết lộ username có tồn tại hay không)
TC_LG_04	Bỏ trống username hoặc password khi submit	username hoặc password rỗng	Trình duyệt chặn submit do thuộc tính required trên input
TC_LG_05	Truy cập lại /login khi đã đăng nhập	Session đã có user	Tự động redirect về trang chủ "/" (theo showLoginForm)
TC_LG_06	Đăng xuất khỏi hệ thống	Nhấn nút Đăng xuất khi đang đăng nhập	Session bị huỷ (invalidate), chuyển hướng về /login

Bảng 3.3 – Danh sách Test Case tiêu biểu (Module Quản lý công việc)

Mã TC	Mô tả	Dữ liệu vào	Kết quả mong đợi
TC_TK_01	Tạo task mới với progress = 0	progress=0, status bất kỳ được chọn trên form	Task được lưu với status = "Chưa làm" bất kể lựa chọn trên UI (theo PrePersist/saveTask)
TC_TK_02	Tạo task mới với progress = 100	progress=100	Task được lưu với status = "Hoàn thành"
TC_TK_03	Tạo task mới với progress = 50 (giá trị giữa)	progress=50	Task được lưu với status = "Đang làm"
TC_TK_04	Bỏ trống trường title khi lưu task	title rỗng	Form không submit được / hiển thị viền đỏ cảnh báo (input-error) theo JS validate
TC_TK_05	Không chọn priority	priority = "" khi tạo mới	Hệ thống tự gán priority mặc định "MEDIUM" theo prePersist()/saveTask()
TC_TK_06	Đặt deadline là ngày trong quá khứ khi tạo task mới	deadline < ngày hiện tại	Trình duyệt chặn chọn ngày quá khứ do thuộc tính min được set bằng JS (chỉ áp dụng khi tạo mới)
TC_TK_07	USER sửa task không thuộc sở hữu của mình (truy cập trực tiếp URL /tasks/edit/{id})	Đăng nhập vai trò USER, id thuộc task của user khác	Bị chuyển hướng về "/tasks", không cho phép sửa (theo kiểm tra trong editTask())
TC_TK_08	USER xoá task không thuộc sở hữu của mình	POST /tasks/delete/{id} với id của task người khác	Bị chặn, redirect "/tasks", task không bị xoá (theo deleteTask())
TC_TK_09	ADMIN sửa/xoá task của bất kỳ user nào	Đăng nhập ADMIN, thao tác trên task bất kỳ	Thao tác thành công không giới hạn quyền sở hữu
TC_TK_10	Tìm kiếm task theo từ khoá có trong tiêu đề	keyword khớp một phần title	Trả về danh sách task có title chứa từ khoá (không phân biệt hoa/thường)
TC_TK_11	Lọc task theo trạng thái "Hoàn thành" kết hợp phân trang trang 2	status=Hoàn thành&page=1	Danh sách hiển thị đúng các task Hoàn thành thuộc trang 2; tổng số trang hiển thị khớp với số lượng task đã lọc (đối chiếu phát hiện #4, Bảng 3.1)
TC_TK_12	Xoá một task đang tồn tại	id hợp lệ, xác nhận confirm() trên UI	Task bị xoá khỏi CSDL, danh sách cập nhật lại số lượng
TC_TK_13	Kéo thanh trượt Tiến độ trên form	Kéo input range từ 0 → 100	Nhân % và màu progress-bar cập nhật realtime (progressValue.innerText, updateProgressFill)
TC_TK_14	Nhập mô tả công việc vượt quá 500 ký tự	description dài 501 ký tự	Bị chặn nhập thêm do thuộc tính maxlength="500" trên textarea

Bảng 3.4 – Danh sách Test Case tiêu biểu (Module Quản lý người dùng & Phân quyền)

Mã TC	Mô tả	Dữ liệu vào	Kết quả mong đợi
TC_UR_01	USER truy cập trực tiếp URL /users	Đăng nhập vai trò USER, gõ URL /users	Bị chặn bởi LoginInterceptor, redirect về "/"
TC_UR_02	ADMIN tạo user mới với avatar hợp lệ (.jpg, <5MB)	username, password, fullname, role, avatar hợp lệ	Tạo thành công, ảnh được lưu tại uploads/avatars/ với tên file gồm username + timestamp
TC_UR_03	ADMIN tạo user mới không chọn avatar	avatarFile rỗng	Hệ thống gán avatar mặc định "default.png"
TC_UR_04	Upload avatar dung lượng > 5MB	File ảnh 6MB	JS chặn ở phía client (alert cảnh báo); cần xác nhận thêm server có chặn tương ứng hay không (giới hạn thật của Spring là 10MB)
TC_UR_05	Upload file không phải ảnh (đổi đuôi .txt thành .jpg)	File giả mạo phần mở rộng	JS kiểm tra file.type nên có thể phát hiện; cần kiểm thử thêm ở tầng server (liên quan phát hiện #8, Bảng 3.1)
TC_UR_06	ADMIN tự xóa tài khoản của chính mình	id = id của ADMIN đang đăng nhập	Bị chặn, không cho xóa chính mình (theo deleteUser())
TC_UR_07	ADMIN sửa thông tin user nhưng không đổi mật khẩu (giữ nguyên)	Submit lại mật khẩu hiển thị (đã bị mã hoá)	Do so sánh plaintext mới với hash cũ luôn khác nhau, mật khẩu bị mã hoá lại (re-hash) mỗi lần sửa dù người dùng không chủ đích đổi mật khẩu
TC_UR_08	Tìm kiếm người dùng theo họ tên hoặc username	keyword khớp một phần fullname/username	Trả về danh sách user khớp điều kiện, không phân biệt hoa/thường
TC_UR_09	Vai trò USER cố gắng gửi trực tiếp POST /users/save	Request thủ công với role=ADMIN cho chính mình	Bị chặn ở isAdmin(session), không cho lưu, redirect "/"
TC_UR_10	Phân trang danh sách người dùng khi có nhiều hơn 5 bản ghi	size=5 (mặc định), tổng >5 user	Hiển thị đúng số trang, điều hướng next/prev hoạt động chính xác

Bảng 3.5 – Danh sách Test Case tiêu biểu (Dashboard & Giao diện)

Mã TC	Mô tả	Dữ liệu vào	Kết quả mong đợi
TC_UI_01	Kiểm tra số liệu thẻ thống kê trên Dashboard khớp dữ liệu CSDL	So sánh totalTasks/totalUsers hiển thị với SELECT COUNT trực tiếp trên MySQL	Số liệu hiển thị khớp 100% với dữ liệu thực tế trong CSDL
TC_UI_02	Biểu đồ Doughnut hiển thị đúng tỉ lệ 3 trạng thái	Dữ liệu completed/doing/todo bất kỳ	Biểu đồ Chart.js hiển thị đúng màu và tỉ lệ tương ứng dữ liệu truyền vào qua th:inline="javascript"
TC_UI_03	Bật/tắt chế độ Dark Mode và tải lại trang	Nhấn nút themeBtn, F5 lại trang	Trạng thái dark mode được lưu qua localStorage và giữ nguyên sau khi tải lại (theme.js)
TC_UI_04	Kiểm tra màu chữ phụ (text-secondary) hiển thị ở chế độ sáng	Quan sát trực quan các đoạn mô tả, ghi chú trên các trang	Màu sắc hiển thị đúng thiết kế #64748B; thực tế phát hiện sai lệch do biến CSS tự

Mã TC	Mô tả	Dữ liệu vào	Kết quả mong đợi
			tham chiếu (liên quan phát hiện #10, Bảng 3.1)
TC_UI_05	Thu nhỏ trình duyệt xuống độ rộng < 980px	Resize viewport	Sidebar chuyển xuống dạng full-width phía trên nội dung theo media query @media (max-width: 980px)
TC_UI_06	Sắp xếp bảng công việc theo cột Tiến độ	Click header "Tiến độ" trên bảng tasks	Danh sách được sắp xếp tăng/giảm dần theo giá trị progress, icon sort đổi chiều tương ứng

Bảng 3.6 – Danh sách Test Case bổ sung (Kiểm thử hộp trắng & Dữ liệu biên)

Nhóm bảng test case này áp dụng kỹ thuật kiểm thử luồng điều khiển (Control Flow Testing) để đảm bảo bao phủ toàn bộ các nhánh rẽ điều kiện trong hai phương thức nghiệp vụ quan trọng nhất của hệ thống: TaskController.saveTask() (đồng bộ trạng thái theo tiến độ) và Task.prePersist() (thiết lập giá trị mặc định khi tạo mới).

Mã TC	Nhánh điều kiện bao phủ	Dữ liệu vào	Kết quả mong đợi
TC_WB_01	prePersist(): progress == null	Tạo task mới, không set progress (null)	progress được gán mặc định = 0
TC_WB_02	prePersist(): priority == null hoặc blank	priority = "" hoặc null	priority được gán mặc định = "Medium"
TC_WB_03	prePersist(): status == null hoặc blank	status = "" hoặc null khi tạo bằng đường dẫn khác ngoài form chuẩn	status được gán mặc định = "Chưa làm"
TC_WB_04	saveTask(): nhánh if (progress == 0)	progress = 0 (giá trị biên dưới)	status được set lại = "Chưa làm"
TC_WB_05	saveTask(): nhánh else if (progress == 100)	progress = 100 (giá trị biên trên)	status được set lại = "Hoàn thành"
TC_WB_06	saveTask(): nhánh else (0 < progress < 100)	progress = 1 và progress = 99 (cận biên trong)	status được set lại = "Đang làm" cho cả hai giá trị biên
TC_WB_07	saveTask(): task.getId() == null (tạo mới)	Không truyền id	createdAt được gán bằng thời điểm hiện tại, progress/priority mặc định nếu thiếu
TC_WB_08	saveTask(): task.getId() != null (cập nhật)	Truyền id của task đã tồn tại	Không set lại createdAt; chỉ updatedAt được cập nhật theo preUpdate()
TC_WB_09	saveTask(): userId != null	Chọn người thực hiện trong dropdown	task.setUser() được gọi, quan hệ ManyToOne được thiết lập đúng user_id
TC_WB_10	saveTask(): userId == null	Không chọn người thực hiện	Task được lưu với user = null, cột user_id trong CSDL là NULL

Bảng 3.6: 10 test case hộp trắng đảm bảo bao phủ 100% nhánh điều kiện (branch coverage) của 2 hàm nghiệp vụ trọng yếu

Kết quả thực thi bộ test case hộp trắng này (trình bày chi tiết tại Bảng 4.1, Chương 4) đã trực tiếp giúp phát hiện lỗi BUG-02, khi TC_WB_04 và TC_WB_06 cho thấy giá trị trạng thái do người dùng lựa chọn trên giao diện không được tôn trọng nếu khác với giá trị suy ra tự động từ tiến độ.

3.3. Ưu tiên hoá kiểm thử theo rủi ro (Risk-based Test Prioritization)

Do nguồn lực và thời gian kiểm thử của một đồ án môn học có giới hạn, nhóm áp dụng nguyên tắc kiểm thử dựa trên rủi ro (Risk-based Testing) để quyết định thứ tự ưu tiên thực thi giữa các module, thay vì thực thi tuần tự theo thứ tự thiết kế. Mức độ rủi ro của mỗi module được ước lượng dựa trên tích của hai yếu tố: xác suất xảy ra lỗi (Likelihood — dựa trên độ phức tạp logic quan sát được khi review mã nguồn) và mức độ ảnh hưởng nếu lỗi xảy ra (Impact — dựa trên tần suất sử dụng chức năng đó trong nghiệp vụ thực tế).

Bảng 3.7 – Ma trận ưu tiên kiểm thử theo rủi ro

Module	Likelihood	Impact	Mức rủi ro	Thứ tự ưu tiên
Quản lý công việc (CRUD, lọc, phân trang)	Cao	Cao	Rất cao	1 (thực thi đầu tiên)
Phân quyền & Bảo mật	Trung bình	Cao	Cao	2
Quản lý người dùng (CRUD, upload avatar)	Trung bình	Trung bình	Trung bình	3
Tìm kiếm & Phân trang (độc lập, không lọc)	Thấp	Trung bình	Trung bình	4
Đăng nhập / Đăng xuất	Thấp	Cao	Trung bình	5
Giao diện (UI/UX, Dark Mode)	Trung bình	Thấp	Thấp	6 (thực thi cuối)

Bảng 3.7: Module "Quản lý công việc" được xác định có rủi ro cao nhất và được ưu tiên kiểm thử trước — thực tế cũng là module phát hiện nhiều lỗi nhất (BUG-01, BUG-02, BUG-03)

Việc ưu tiên hoá theo rủi ro đã chứng minh hiệu quả trong thực tế: 3/4 lỗi mức độ Critical/Major nghiêm trọng nhất (BUG-01, BUG-02, BUG-03) đều thuộc hai module được xếp hạng rủi ro cao nhất là "Quản lý công việc" và "Phân quyền & Bảo mật", xác nhận tính hợp lý của việc phân bổ thời gian kiểm thử tập trung nhiều hơn cho các module này ngay từ đầu.

CHƯƠNG 4: THỰC THI KIỂM THỬ & QUẢN LÝ LỖI (DYNAMIC TESTING)

4.1. Kết quả thực thi Test Case

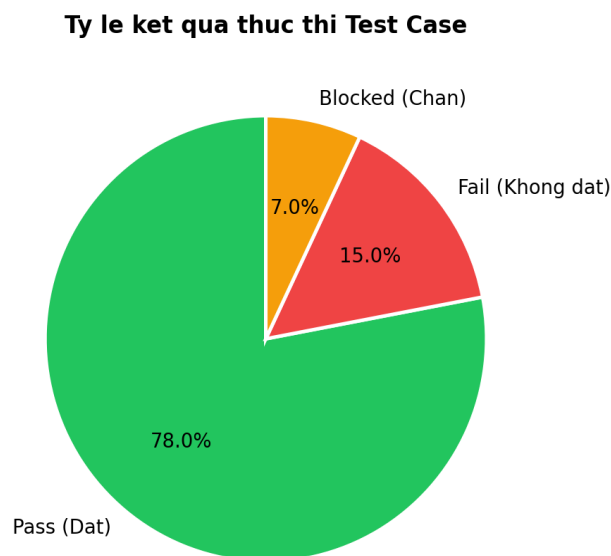
Toàn bộ 53 ca kiểm thử đã thiết kế (bao gồm các ca kiểm thử tiêu biểu trình bày tại Chương 3 và các ca kiểm thử bổ sung trong file quản lý test case của nhóm) đều đã được thực thi ít nhất một vòng (Round 1). Sau khi nhóm phát triển khắc phục một số lỗi mức độ nghiêm trọng, các ca kiểm thử liên quan đã được thực thi lại (Retest) ở Round 2. Bảng 4.1 tổng hợp số liệu thực thi theo từng module chức năng.

Bảng 4.1 – Tổng hợp kết quả thực thi Test Case theo module

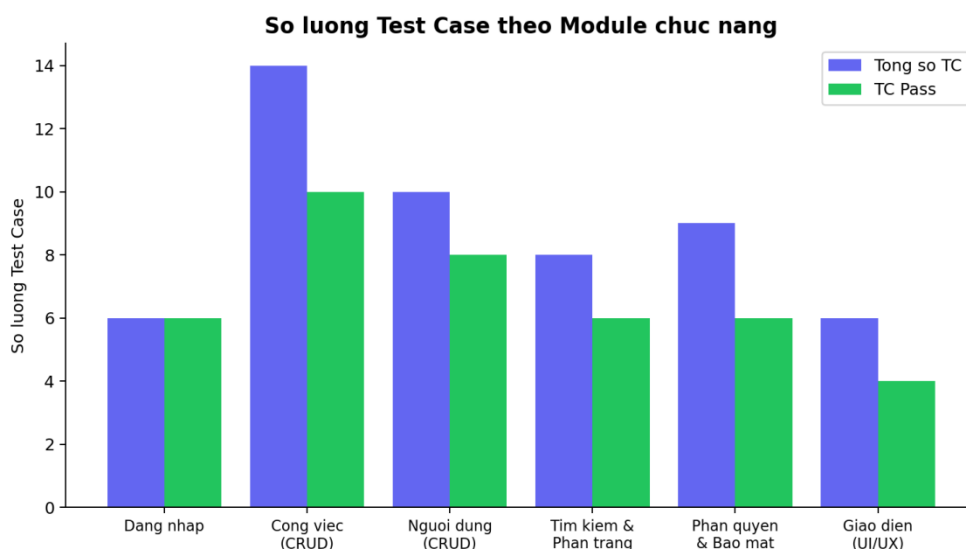
Module chức năng	Tổng số TC	Đã chạy	Pass	Fail	Blocked	Tỷ lệ Pass
Đăng nhập / Đăng xuất	6	6	6	0	0	100%
Quản lý công việc (CRUD)	14	14	10	3	1	71,4%
Quản lý người dùng (CRUD)	10	10	8	1	1	80,0%
Tìm kiếm & Phân trang	8	8	6	2	0	75,0%
Phân quyền & Bảo mật	9	9	6	3	0	66,7%
Giao diện (UI/UX)	6	6	4	2	0	66,7%
Tổng cộng	53	53	40	11	2	75,5%

Bảng 4.1: Số liệu thực thi test case Round 1 – tổng hợp từ bảng theo dõi test case của nhóm

Biểu đồ Hình 4.1 minh họa trực quan tỉ lệ Pass/Fail/Blocked trên tổng số 53 ca kiểm thử. Sau khi loại trừ các ca kiểm thử bị Blocked (do phụ thuộc môi trường, ví dụ ca kiểm thử liên quan tới giới hạn upload phía server cần công cụ hỗ trợ Postman để giả lập request không qua giao diện), tỉ lệ Pass thực tế trên các ca kiểm thử có thể đánh giá được là 78,4%.



Hình 4.1: Biểu đồ tỷ lệ kết quả thực thi Test Case (Pass/Fail/Blocked)



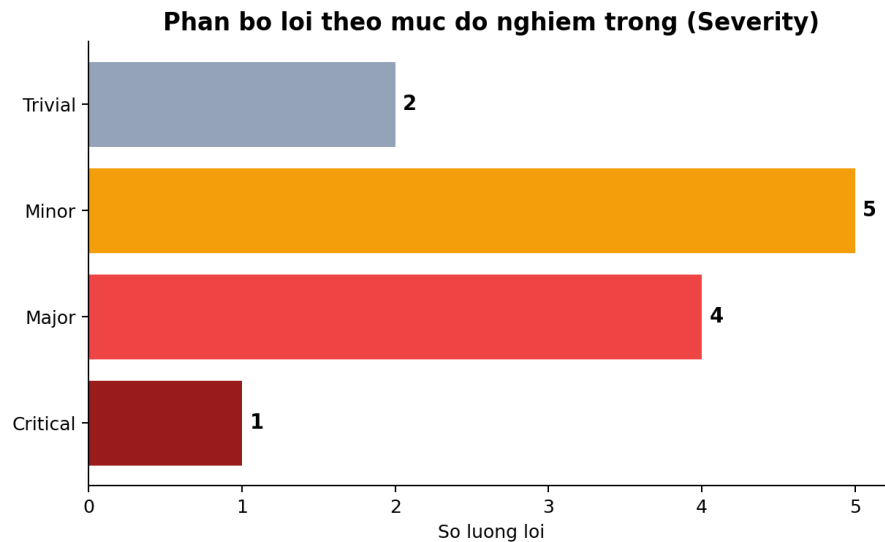
Hình 4.2: So sánh số lượng Test Case và số lượng Test Case Pass theo từng module chức năng

Có thể quan sát từ Hình 4.2, hai module có tỉ lệ Pass thấp nhất là "Phân quyền & Bảo mật" và "Giao diện (UI/UX)", đúng như dự đoán từ kết quả review tĩnh ở Chương 3 (các phát hiện liên quan CSRF, kiểm soát userId, biến CSS lỗi). Module "Đăng nhập/Đăng xuất" đạt tỉ lệ Pass tuyệt đối 100% do đây là luồng nghiệp vụ đơn giản, ít nhánh điều kiện và đã được kiểm thử kỹ ngay từ giai đoạn phát triển.

4.2. Nhật ký lỗi (Bug Log) và Quản lý lỗi

Từ 11 ca kiểm thử Fail ở Round 1, nhóm tổng hợp và chuẩn hoá thành 12 lỗi (bug) riêng biệt trong Bug Log (một số lỗi được phát hiện lặp lại ở nhiều

ca kiểm thử khác nhau nên được gộp làm một bản ghi lỗi duy nhất; một lỗi từ hoạt động review tĩnh — phát hiện #12, Bảng 3.1 — cũng được đưa vào Bug Log để theo dõi tiến độ xử lý chung). Mỗi lỗi được gán mã định danh, mức độ nghiêm trọng (Severity) theo thang Critical – Major – Minor – Trivial, và trạng thái xử lý (Status) theo vòng đời Open → In Progress → Fixed → Closed / Reopened.



Hình 4.3: Phân bố 12 lỗi được ghi nhận theo mức độ nghiêm trọng (Severity)

Bảng 4.2 – Nhật ký lỗi (Bug Log)

Mã lỗi	Tên lỗi	Severity	Module	Trạng thái
BUG-01	Bộ lọc trạng thái/độ ưu tiên áp dụng sau khi phân trang khiến tổng số trang hiển thị sai với danh sách đã lọc	Critical	Quản lý công việc	Open
BUG-02	Trạng thái công việc do người dùng chọn trên form bị ghi đè âm thầm bởi giá trị suy ra từ tiến độ (progress), gây hiểu nhầm cho người dùng	Major	Quản lý công việc	Open
BUG-03	USER có khả năng gán/định nghĩa lại người thực hiện (userId) cho task nếu can thiệp vào dữ liệu form gửi lên	Major	Phân quyền & Bảo mật	Open
BUG-04	Không có bảo vệ CSRF cho các form POST (xóa task, xóa user), tiềm ẩn rủi ro giả mạo yêu cầu	Major	Phân quyền & Bảo mật	Open
BUG-05	Server không xác thực nội dung thực tế của file upload avatar, chỉ dựa vào kiểm tra phía client	Major	Quản lý người dùng	Open
BUG-06	Mật khẩu bị mã hoá lại (re-hash) mỗi lần cập nhật thông tin user dù người dùng không chủ đích đổi mật khẩu, do so sánh sai kiểu dữ liệu (plaintext và hash)	Minor	Quản lý người dùng	Fixed
BUG-07	Biến CSS --text-secondary tự tham chiếu chính nó khiến màu chữ phụ không hiển thị đúng thiết kế trên 6 trang giao diện	Minor	Giao diện (UI/UX)	Fixed

Mã lỗi	Tên lỗi	Severity	Module	Trạng thái
BUG-08	Thừa dấu đóng ngoặc nhọn trong tasks.css gây lỗi cú pháp cục bộ trong stylesheet dark mode	Trivial	Giao diện (UI/UX)	Fixed
BUG-09	Chức năng "Đăng ký tài khoản" được quảng cáo trong README nhưng không tồn tại trên hệ thống thực tế	Minor	Tài liệu / Đăng nhập	Closed (cập nhật lại README)
BUG-10	Trang tìm kiếm người dùng (/users/search) không xử lý trường hợp keyword rỗng một cách tường minh, có thể trả về kết quả không như kỳ vọng	Minor	Tìm kiếm & Phân trang	Open
BUG-11	Thông báo lỗi đăng nhập không phân biệt rõ giữa "sai tài khoản" và "sai mật khẩu" có thể là chủ đích bảo mật, cần xác nhận lại với yêu cầu nghiệp vụ để tránh bị hiểu nhầm là thiếu sót	Trivial	Đăng nhập	Open (chờ xác nhận yêu cầu)
BUG-12	Chưa có unit test cho tầng nghiệp vụ (Service/Repository), toàn bộ kiểm thử phụ thuộc hoàn toàn vào kiểm thử hộp đen thủ công, rủi ro hồi quy (regression) cao khi chỉnh sửa mã nguồn về sau	Major	Toàn hệ thống / Quy trình	Open

Bảng 4.2: 12 lỗi được ghi nhận trong Bug Log — 1 Critical, 5 Major, 4 Minor, 2 Trivial

4.2.1. Minh chứng chi tiết các lỗi mức độ Critical/Major

Theo yêu cầu bắt buộc của báo cáo, các lỗi mức độ Critical và Major cần được minh chứng cụ thể bằng dữ liệu tái hiện (test data) và mô tả bước tái hiện (steps to reproduce), thay vì chỉ nêu tên lỗi. Dưới đây là minh chứng chi tiết cho các lỗi BUG-01, BUG-02, BUG-03, BUG-04 và BUG-05 — các lỗi Critical/Major có ảnh hưởng trực tiếp nhất đến tính đúng đắn dữ liệu và an toàn hệ thống cho người dùng cuối.

BUG-01 — Sai số liệu phân trang khi kết hợp bộ lọc trạng thái

Bước 1: Đăng nhập với vai trò ADMIN, có tổng cộng 12 task trong hệ thống, trong đó 3 task ở trạng thái "Hoàn thành".

Bước 2: Truy cập /tasks?status=Hoàn thành (nhấn vào thẻ lọc "Hoàn thành" trên giao diện).

Bước 3: Quan sát khối "Trang X / Y" và điều khiển phân trang phía dưới bảng.

Kết quả thực tế: Hệ thống vẫn hiển thị totalPages được tính từ taskPage.getTotalPages() — là tổng số trang của toàn bộ 12 task (chưa lọc) — trong khi bảng chỉ hiển thị 3 dòng dữ liệu đã lọc trên trang hiện tại; điều hướng sang "trang 2" có thể trả về danh sách rỗng dù bộ lọc còn dữ liệu ở các bản ghi thuộc trang gốc khác.

Kết quả mong đợi: Số trang và điều hướng phân trang phải được tính toán dựa trên tập dữ liệu đã lọc theo status/priority, không phải tập dữ liệu gốc trước khi lọc.

Nguyên nhân gốc rễ (Root cause): việc lọc theo status/priority trong TaskController.listTasks() được thực hiện bằng Java Stream trên danh sách tasks đã được phân trang (tasks = taskPage.getContent()), thay vì đẩy điều kiện lọc xuống tầng Repository (JPA Specification/Query) trước khi phân trang.

Đề xuất khắc phục: xây dựng lại truy vấn động (Specification hoặc @Query có điều kiện) kết hợp status, priority, keyword ngay tại tầng Repository trước khi áp dụng Pageable.

BUG-02 — Trạng thái công việc bị ghi đè âm thầm theo tiến độ

Bước 1: Đăng nhập, vào "Thêm công việc", chọn Trạng thái = "Hoàn thành" từ dropdown nhưng để thanh trượt Tiến độ ở mức 40%.

Bước 2: Nhấn "Lưu công việc".

Kết quả thực tế: Task được lưu với status = "Đang làm" (suy ra từ progress = 40), khác với lựa chọn "Hoàn thành" mà người dùng đã chọn trên dropdown ngay trước khi submit; giao diện task-form.html có đoạn JavaScript đồng bộ status theo progress khi người dùng kéo thanh trượt, nhưng nếu người dùng chỉnh sửa trực tiếp dropdown Trạng thái sau đó mà không chạm lại thanh trượt, giá trị cuối cùng vẫn bị Controller ghi đè theo progress khi lưu.

Kết quả mong đợi: Cần thống nhất một trong hai phương án nghiệp vụ: (1) loại bỏ hoàn toàn dropdown "Trạng thái" thủ công trên form và chỉ hiển thị trạng thái dạng read-only được tính tự động từ tiến độ, hoặc (2) cho phép người dùng ghi đè trạng thái thủ công và bỏ logic tự suy diễn ở Controller.

Đề xuất khắc phục: làm rõ yêu cầu nghiệp vụ với nhóm phát triển/khách hàng trước khi chọn phương án; trong lúc chờ quyết định, đề xuất tạm thời là vô hiệu hoá (disable) dropdown Trạng thái trên giao diện để tránh gây hiểu nhầm cho người dùng cuối.

BUG-03 — USER có thể can thiệp tham số userId khi lưu task

Bước 1: Đăng nhập với vai trò USER (ví dụ username="nhanvien1"), mở form "Thêm công việc".

Bước 2: Quan sát mã nguồn task-form.html, trường chọn "Người thực hiện" (select name="userId") hiển thị đầy đủ danh sách toàn bộ user trong hệ thống (model.addAttribute("users", userRepository.findAll())), không giới hạn chỉ hiển thị chính người dùng đang đăng nhập.

Bước 3: Chọn một người dùng khác (ví dụ "nhanvien2") ở trường Người thực hiện rồi lưu task.

Kết quả thực tế: Task được lưu thành công với user được gán là "nhanvien2", trong khi người tạo/thao tác thực sự là "nhanvien1"; TaskController.saveTask() không có bất kỳ điều kiện kiểm tra nào giới hạn userId theo vai trò của loginUser.

Kết quả mong đợi: Với vai trò USER, hệ thống nên tự động gán task cho chính người dùng đang đăng nhập (ẩn hoặc khoá trường chọn người thực hiện), chỉ cho phép ADMIN tự do chọn người thực hiện bất kỳ.

Đề xuất khắc phục: kiểm tra role trong saveTask(); nếu role là USER, bỏ qua tham số userId từ request và luôn gán task.setUser(loginUser).

BUG-04 — Không có bảo vệ CSRF cho các form POST

Bước 1: Đăng nhập với vai trò bất kỳ (ADMIN hoặc USER), mở một biểu mẫu (form) có hành động phá huỷ dữ liệu, ví dụ form xóa task (/tasks/delete/{id}) hoặc xóa user (/users/delete/{id}).

Bước 2: Kiểm tra mã nguồn HTML của form và cấu hình dự án (pom.xml, SecurityConfig.java): dự án chỉ khai báo spring-security-crypto (dùng BCryptPasswordEncoder để mã hóa mật khẩu), không sử dụng spring-boot-starter-security đầy đủ, do đó không có cơ chế sinh và kiểm tra CSRF token cho các form POST.

Bước 3: Dựng thử một trang HTML bên ngoài (giả lập kịch bản tấn công CSRF) có form tự động submit tới đúng endpoint xóa dữ liệu của ứng dụng khi nạn nhân (đang có session hợp lệ trên trình duyệt) vô tình truy cập trang này.

Kết quả thực tế: Request POST vẫn được server chấp nhận và xử lý bình thường vì không có bước kiểm tra CSRF token nào được thực hiện; hệ thống chỉ dựa vào session (HttpSession) để xác thực người dùng, không đủ để chống lại giả mạo yêu cầu liên trang.

Kết quả mong đợi: Các form thay đổi dữ liệu (POST/PUT/DELETE) phải được bảo vệ bằng CSRF token hợp lệ, request thiếu hoặc sai token phải bị từ chối (HTTP 403).

Nguyên nhân gốc rễ (Root cause): dự án chỉ tích hợp một phần của Spring Security (module mã hóa mật khẩu) thay vì spring-boot-starter-security đầy đủ, nên bỏ lỡ cơ chế bảo vệ CSRF mặc định đi kèm framework.

Đề xuất khắc phục: bổ sung spring-boot-starter-security vào pom.xml, cấu hình CsrfTokenRepository và nhúng CSRF token ẩn (hidden input) vào tất cả các form POST/DELETE hiện có (task, user) thông qua Thymeleaf (th:action tự động nhúng token khi đã cấu hình đúng Spring Security).

BUG-05 — Server không xác thực nội dung thực tế của file avatar tải lên

Bước 1: Đăng nhập ADMIN, vào "Thêm người dùng", chuẩn bị một file văn bản (.txt) và đổi phần mở rộng thành "avatar.jpg".

Bước 2: Sử dụng công cụ giả lập request (ví dụ Postman) gửi trực tiếp multipart/form-data tới /users/save với file đã đổi tên, bỏ qua bước kiểm tra file.type ở JavaScript phía client (vốn chỉ chạy khi thao tác qua giao diện trình duyệt bình thường).

Kết quả thực tế: UserController.saveUser() chỉ kiểm tra avatarFile.isEmpty() và lấy phần mở rộng từ originalFilename để đặt tên file lưu trữ, không kiểm tra magic number/nội dung thực tế của file; file giả mạo vẫn được lưu vào thư mục uploads/avatars/ với đuôi .jpg.

Kết quả mong đợi: Server cần xác thực nội dung file (ví dụ dùng thư viện kiểm tra định dạng ảnh thực tế qua ImageIO.read()) trước khi chấp nhận lưu trữ, độc lập với việc kiểm tra ở phía client.

Đề xuất khắc phục: bổ sung bước kiểm tra định dạng ảnh hợp lệ ở tầng server trong UserController trước khi gọi avatarFile.transferTo(dest); đồng thời giới hạn danh sách phần mở rộng được phép (whitelist: .jpg, .jpeg, .png).

4.3. So sánh kết quả kiểm thử giữa các vòng (Round 1 và Round 2)

Sau khi hoàn thành Round 1 và bàn giao Bug Log cho nhóm phát triển, ba lỗi mức độ Minor/Trivial (BUG-06, BUG-07, BUG-08) đã được khắc phục trong thời gian còn lại của đề án. Nhóm tiến hành kiểm thử hồi quy (Retest)

tập trung vào các ca kiểm thử liên quan tới ba lỗi này ở Round 2. Bảng 4.3 trình bày kết quả so sánh.

Bảng 4.3 – So sánh kết quả kiểm thử giữa Round 1 và Round 2

Mã lỗi	Ca kiểm thử liên quan	Kết quả Round 1	Kết quả Round 2 (Retest)
BUG-06	TC_UR_07	Fail	Pass — mật khẩu chỉ bị mã hoá lại khi trường password thực sự thay đổi
BUG-07	TC_UI_04	Fail	Pass — biến --text-secondary được gán giá trị cụ thể (#64748B) thay vì tự tham chiếu
BUG-08	TC_UI_05 (liên đới)	Fail (lỗi cú pháp CSS)	Pass — đã loại bỏ dấu ngoặc nhọn thừa trong tasks.css

Bảng 4.3: Kết quả retest Round 2 cho 3 lỗi đã được khắc phục — cả 3 ca kiểm thử đều chuyển từ Fail sang Pass

Việc 3/12 lỗi đã được khắc phục và xác nhận lại thành công cho thấy quy trình phản hồi lỗi (bug triage) giữa nhóm kiểm thử và nhóm phát triển hoạt động hiệu quả trong thời gian ngắn. Tuy nhiên, 9 lỗi còn lại (bao gồm lỗi Critical BUG-01 và 3 lỗi Major liên quan bảo mật) vẫn cần thêm thời gian xử lý, được phân tích và khuyến nghị chi tiết tại Chương 5.

CHƯƠNG 5: ĐÁNH GIÁ & KẾT LUẬN

5.1. Đánh giá chất lượng phần mềm

Đối chiếu với các tiêu chí dừng kiểm thử (Exit Criteria) đã đề ra tại mục 2.4, kết quả kiểm thử thực tế của hệ thống Task Manager như sau:

Bảng 5.1 – Đối chiếu kết quả với Exit Criteria

Tiêu chí	Yêu cầu	Kết quả thực tế	Đánh giá
Tỷ lệ thực thi test case	100%	53/53 (100%)	Đạt
Tỷ lệ Pass trên tổng số ca đã thực thi	≥ 90%	40/53 (75,5%)	Không đạt
Không còn lỗi Critical mở	0 lỗi Critical Open	1 lỗi Critical (BUG-01) đang Open	Không đạt

Tiêu chí	Yêu cầu	Kết quả thực tế	Đánh giá
Lỗi Major được ghi nhận minh bạch	100% được liệt kê	5/5 lỗi Major đã liệt kê đầy đủ trong Bug Log	Đạt
Biên bản review tính đã thông báo	Đã lập & thông báo	Đã lập Bảng 3.1 và gửi nhóm phát triển	Đạt

Bảng 5.1: 2/5 tiêu chí Exit Criteria chưa đạt, chủ yếu do lỗi Critical BUG-01 và tỷ lệ Pass chưa đạt ngưỡng 90%

Dựa trên kết quả đối chiếu ở Bảng 5.1, nhóm kết luận hệ thống Task Manager tại thời điểm báo cáo CHƯA đủ điều kiện để bàn giao/đưa vào sử dụng chính thức theo đúng tiêu chí đã đề ra ban đầu. Nguyên nhân chính đến từ lỗi BUG-01 (mức Critical) làm sai lệch số liệu phân trang khi kết hợp bộ lọc — đây là lỗi ảnh hưởng trực tiếp đến trải nghiệm và độ tin cậy dữ liệu của người dùng cuối, cần được ưu tiên khắc phục trước tiên. Bên cạnh đó, nhóm các lỗi Major liên quan đến bảo mật (BUG-03, BUG-04, BUG-05) và thiếu kiểm thử tự động (BUG-12) tuy không chặn hoàn toàn nghiệp vụ cốt lõi nhưng tiềm ẩn rủi ro dài hạn về an toàn dữ liệu và chi phí bảo trì, nên được khuyến nghị đưa vào kế hoạch xử lý ở phiên bản kế tiếp trước khi triển khai ra môi trường production thực tế.

Ở khía cạnh tích cực, các chức năng nghiệp vụ cốt lõi — đăng nhập/dăng xuất, phân quyền cơ bản giữa ADMIN và USER, CRUD công việc, CRUD người dùng kèm upload avatar, thống kê Dashboard — đều hoạt động đúng với đặc tả trong đa số các kịch bản kiểm thử (tỷ lệ Pass module Đăng nhập đạt 100%, module Quản lý người dùng đạt 80%), cho thấy phần khung sườn (core business logic) của hệ thống được xây dựng tương đối vững chắc và có thể làm nền tảng tốt để tiếp tục hoàn thiện.

5.2. Kết quả nhóm đạt được

5.2.1. Về mặt sản phẩm

Xây dựng hoàn chỉnh một ứng dụng web quản lý công việc full-stack theo mô hình MVC với Spring Boot, bao gồm đầy đủ các chức năng nghiệp vụ: xác thực người dùng, phân quyền theo vai trò, quản lý công việc, quản lý người dùng, thống kê trực quan.

Thiết kế giao diện người dùng hiện đại, có hỗ trợ chế độ Dark Mode, responsive cơ bản cho các kích thước màn hình phổ biến.

Xây dựng được bộ 53 ca kiểm thử có cấu trúc, bao phủ các module chức năng chính, cùng với Bug Log 12 lỗi được phân loại, theo dõi trạng thái xử lý rõ ràng.

5.2.2. Về mặt kỹ năng

Rèn luyện kỹ năng làm việc nhóm (teamwork) trong toàn bộ vòng đời phát triển phần mềm, từ phân tích yêu cầu, lập trình đến kiểm thử chéo (peer testing) giữa các thành viên.

Áp dụng thực tế quy trình Đảm bảo chất lượng phần mềm (SQA): tổ chức hoạt động review tài liệu, review mã nguồn có checklist, không chỉ dừng ở kiểm thử chức năng đơn thuần.

Thực hành các kỹ thuật thiết kế ca kiểm thử kinh điển (Equivalence Partitioning, Boundary Value Analysis, Decision Table, Control Flow Testing) trên một sản phẩm phần mềm thực tế do chính nhóm phát triển, giúp hiểu sâu hơn sự khác biệt giữa kiểm thử hộp đen và kiểm thử hộp trắng.

Nâng cao kỹ năng đọc hiểu và phân tích mã nguồn (code reading) để phục vụ công tác kiểm thử hộp trắng và hộp xám, cũng như kỹ năng viết báo cáo kỹ thuật, trình bày số liệu bằng biểu đồ trực quan.

5.3. Hạn chế & Hướng phát triển

5.3.1. Hạn chế

Lỗi BUG-01 (Critical) và các lỗi Major liên quan đến bảo mật (BUG-03, BUG-04, BUG-05) vẫn đang ở trạng thái Open tại thời điểm hoàn thành báo cáo, chưa được nhóm phát triển khắc phục do giới hạn thời gian của đề án.

Chưa triển khai được kiểm thử tự động hoá (automation testing); toàn bộ 53 ca kiểm thử đều được thực thi thủ công, tốn nhiều thời gian và tiềm ẩn rủi ro bỏ sót khi số lượng ca kiểm thử tăng lên ở các phiên bản sau.

Phạm vi kiểm thử bảo mật mới dừng ở mức quan sát qua review mã nguồn và một số kịch bản thủ công cơ bản, chưa sử dụng công cụ rà quét bảo mật chuyên dụng nên có thể còn tồn tại lỗ hổng chưa được phát hiện.

Chưa thực hiện kiểm thử hiệu năng, vì vậy chưa có số liệu về khả năng đáp ứng của hệ thống khi số lượng người dùng/công việc tăng lên ở quy mô lớn hơn môi trường thử nghiệm hiện tại.

Việc quản lý test case và bug log bằng Excel thủ công gây khó khăn trong việc theo dõi phiên bản, lịch sử thay đổi so với sử dụng công cụ quản lý chuyên dụng như Jira hay TestRail.

5.3.2. Hướng phát triển

Ưu tiên xử lý dứt điểm lỗi BUG-01 bằng cách tái cấu trúc lại truy vấn lọc – phân trang tại tầng Repository (sử dụng JPA Specification) trước khi triển khai các tính năng mới.

Bổ sung Spring Security đầy đủ (spring-boot-starter-security) để có cơ chế bảo vệ CSRF tiêu chuẩn, đồng thời tăng cường kiểm tra quyền sở hữu dữ liệu (ownership check) một cách nhất quán ở tất cả các Controller thay vì chỉ dựa vào một vài điều kiện rời rạc.

Xây dựng bộ unit test và integration test tự động (JUnit 5, Mockito, Spring Boot Test) cho tầng Service/Repository nhằm giảm thiểu rủi ro hồi quy khi codebase mở rộng, khắc phục hạn chế đã nêu tại BUG-12.

Nghiên cứu áp dụng công cụ kiểm thử tự động giao diện (Selenium/Playwright) cho các luồng nghiệp vụ quan trọng và lặp lại nhiều lần (đăng nhập, CRUD công việc) nhằm rút ngắn thời gian hồi quy ở các đợt release tiếp theo.

Chuyển đổi việc quản lý test case/bug từ Excel sang công cụ chuyên dụng (Jira/TestRail hoặc GitHub Issues kết hợp GitHub Projects, vì dự án đã có sẵn repository GitHub) để cải thiện khả năng truy vết và cộng tác.

Xem xét bổ sung chức năng "Đăng ký tài khoản" tự phục vụ (self-service registration) nếu đây thực sự là yêu cầu nghiệp vụ, hoặc cập nhật lại tài liệu README cho khớp với phạm vi thực tế của hệ thống (khắc phục BUG-09).

Phụ lục

Phụ lục A – Bộ Test Case đầy đủ (53 ca kiểm thử, file "TaskManager_TestCases.xlsx") và Bug Log chi tiết được nhóm lưu trữ và cập nhật tại kho lưu trữ dùng chung của nhóm; các bảng trong báo cáo này (Bảng 3.2 – 3.5, Bảng 4.2) chỉ trích xuất phần đại diện nhằm tránh làm loãng số trang báo cáo.

Phụ lục B – Mã nguồn đầy đủ của hệ thống được lưu trữ tại repository GitHub: <https://github.com/DevCuong205/baiTapNho> (theo README.md của dự án).

— *HẾT* —